

DOCUMENTO NORMATIVO · ERP DATAFAST

Directrices, Políticas y Filosofía — ERP Datafast

ERP Datafast ISP

Documento normativo · Compilado: 2026-08-06 · Rama `main` , commit base `f8d52b00`

Documento normativo — Directrices, Políticas y Filosofía

Nota preliminar sobre este documento

Este documento **no inventa ninguna regla**. Compila las directrices, políticas y principios que ya gobiernan el ERP Datafast, extraídos de sus fuentes primarias:
Fecha de levantamiento: 2026-08-06 · **Rama:** `main` · **Commit base:** `f8d52b00`
Alcance: compila las reglas vigentes desde sus fuentes primarias. No inventa ninguna regla.

Fuente	Documento: Directrices, Políticas y Filosofía	Qué aporta
<code>CLAUDE.md</code> (raíz del repositorio)		Las reglas de construcción declaradas del proyecto
Comentarios de diseño en el código		Directrices que solo viven ahí y no están en ningún documento
<code>pagos/adaptadores/README.md</code>		La política de la frontera del dinero
<code>ecosystem.config.js</code>		Las políticas de despliegue y aislamiento de procesos
<code>PENDIENTES.md</code>		El criterio de registro de deuda técnica
Tests <code>*.spec.ts</code>		Los invariantes que están verificados , no solo escritos
<code>CLAUDE.md</code> global del usuario		El rol y las directrices de trabajo del equipo

Cada directriz indica **su origen** (el incidente que la motivó, cuando existe), **dónde está codificada** y **cómo se verifica**. Esa última columna es la más importante: separa lo que el sistema *garantiza* de lo que solo *aspira*.

Hallazgo que atraviesa todo el documento: varias de las directrices más valiosas del ERP **solo existen como comentarios dentro de un archivo de código**. No están en `CLAUDE.md` , no están en `docs/` , y nadie que no abra ese archivo concreto sabrá que existen. Se marcan con **⚠ SOLO EN CÓDIGO**.

PARTE I — FILOSOFÍA

Los principios rectores. No son reglas de implementación: son la forma de pensar de la que se derivan todas las reglas.

F1 · El hardware es la verdad; la base de datos es una creencia

En un ERP administrativo la base de datos **es** la realidad. En un ERP de telecomunicaciones, no: la realidad son la OLT, la ONU y el router, y la base de datos es una **afirmación sobre ellos** que puede estar equivocada sin que nada falle.

De este principio se derivan VIO, los watchers de reconciliación, el invariante de atomicidad y la máquina de estados. Es el principio del que cuelga todo el plano de red.

F2 · Aceptar no es aplicar

"Aceptar una configuración no significa que la infraestructura la haya materializado."

Toda operación contra hardware externo tiene **dos estados distintos, y el segundo nunca se asume a partir del primero**:

Estado	Significado
Accepted	El comando CLI/API no devolvió error. Es lo único que confirma un <code>success: true</code> típico. No es suficiente.
Materialized / Verified	Existe evidencia observable, obtenida con un comando de lectura independiente , de que el cambio vive en el plano operativo.

Origen: incidente 2026-07-17 (CNT-2026-000004). Una ONU Huawei EG8145V5 aceptó sin error el comando OMCI del carril TR-069 —la OLT lo mostraba configurado— pero el firmware nunca activó el IP-host: **cero tramas Ethernet emitidas**, confirmado con sniffer durante un cold-boot físico real. El ERP reportó "carril aplicado" durante días mientras la gestión remota estaba completamente muerta, porque el código solo verificaba que el CLI no devolviera error.

F3 · El software también afirma sin verificar

VIO no aplica solo al hardware externo. Aplica a lo que el sistema afirma **sobre sí mismo**.

"Una afirmación sobre el propio sistema es una afirmación **sin verificar** hasta que un test la demuestra. Un comentario que garantiza una propiedad es un `success: true` sin comprobar; la única diferencia es que el driver de hardware al menos tiene la regla escrita."

Origen: `outbox-red.service.ts` afirmaba en un comentario que "`SELECT FOR UPDATE SKIP LOCKED` : dos instancias PM2 nunca toman el mismo registro". **Era falso** — la transacción se cerraba antes de ejecutar contra el hardware. Nadie lo verificó nunca; lo verificó producción, y lo salvó por casualidad un lock que existía por otra razón.

F4 · Causa raíz antes que parche

"Un parche que hace desaparecer el síntoma deja el defecto vivo y, peor, lo deja invisible: el siguiente que lo encuentre partirá de un sistema que 'ya se revisó'. El coste de buscar la raíz se paga una vez; el de no buscarla se paga en cada

aparición."

Origen: 2026-08-05, el mapa de red. Tres fallos encadenados, cada uno con un parche superficial disponible y tentador:

Síntoma	Parche superficial disponible	Causa raíz real
Los abonados no aparecían	Copiar coordenadas a <code>clientes.latitud</code> con un <code>UPDATE</code>	El dato vivía en dos sitios y la consulta leía el equivocado. El <code>UPDATE</code> habría funcionado ese día y fallado con cada alta nueva
Ninguna capa vectorial se dibujaba	Quitar las capas de etiquetas, que parecían sospechosas	<code>maplibre-gl</code> 6.1.0 no procesa GeoJSON bajo el empaquetado de Next 14. Quitar etiquetas no habría cambiado nada y habría enterrado la pista
El GPS del móvil decía "permiso denegado"	Mejorar el texto del mensaje	La cabecera <code>Permissions-Policy</code> declaraba <code>geolocation=()</code> —lista vacía, que prohíbe la ubicación al propio sitio—. Ningún texto habría arreglado eso

F5 · El mecanismo vence a la disciplina

Un invariante que depende de que alguien se acuerde, no es un invariante.

"El invariante que solo vive en la doc no es invariante."

Evidencia acumulada en el propio sistema:

- El índice UNIQUE parcial resolvió los routers duplicados que la revisión manual no evitaba.
- `frontera-dinero.spec.ts` impide la cuarta copia del `UPDATE` que la convención no impidió.
- Un trigger mantiene el saldo aunque el escritor sea un script manual.

F6 · Implementación desde cero

"El ERP inyecta SU configuración canónica en cualquier equipo —nuevo o en producción— y NUNCA se adapta a la preexistente: solo la respeta como intocable."

Verificar no-colisión, bloquear el recurso, **jamás reutilizar**. Un equipo que ya funcionaba se **adopta** (se observa y se respeta), nunca se reconfigura.

Codificado en: `capability/olt-baseline-standard.ts` ⚠ **SOLO EN CÓDIGO**

"el ERP inyecta SU configuración en cualquier OLT — nueva o en producción — y NUNCA se adapta a la preexistente (solo la respeta como intocable). Esta constante es la única fuente de verdad de esa configuración."

F7 · Reutilizar antes de construir

"Dos caminos hacia el mismo dato son dos verdades que empiezan idénticas y divergen en la primera modificación que alguien haga en una sola de ellas — y entonces el ERP responde distinto según por dónde se le pregunte, que es peor que no responder."

Origen: 2026-08-05. Se estuvo a punto de proponer una clasificación de estados de ONU desde cero. **Ya existía:**

`clasificarOnus` lee la OLT por SSH, cruza `display ont info` con la causa de caída y distingue `online` | `apagada` | `ruptura_fibra` | `desactivada` | `offline` — incluido el `ruptura_fibra` que se había dado por imposible de determinar.

Matiz obligatorio: antes de reutilizar, comprobar que **el coste encaja con el uso**. Leer el estado en vivo de una ONU está bien al abrir una ficha, y **tumba la gestión del MA5800** si se hace por cada pin del mapa en cada movimiento. Reutilizar no significa ignorar el patrón de acceso.

F8 · Un fallo silencioso es peor que uno ruidoso

Origen: 2026-08-06, tres veces el mismo día.

"Una verificación que solo sabe confirmar el caso bueno no es una verificación."

`scripts/update.sh` recargaba `--only datafast-backend`, un proceso que **ya no existe**. PM2 no encontraba nada, no fallaba de forma detectable, y el script imprimía «Backend recargado» igual. Las migraciones **sí** corrían —son un paso aparte—, así que **la base de datos avanzaba y el código no**: el esquema decía una cosa y el proceso vivo entendía otra. Durante 11 horas.

F9 · Vocabulario de dominio, no de transporte

Los mismos métodos los consume un humano (controller HTTP) y una máquina (outbox). Escribir los veredictos como excepciones HTTP obliga al reintentador automático a hacer **arqueología sobre códigos de estado**, y se equivoca.

F10 · Lo no confirmado se anula; lo confirmado jamás se anula por un cierre

La frontera de confirmación es el **estado terminal verificado**, no el clic del operador.

PARTE II — DIRECTRICES OBLIGATORIAS DE CONSTRUCCIÓN

Reglas que condicionan cómo se escribe código nuevo. Todas están declaradas en `CLAUDE.md`.

D1 · Módulos degradables: nacen degradados

Enunciado: todo módulo nuevo que dependa de hardware físico, API externa, servicio de terceros o infraestructura opcional **DEBE implementar el patrón degradado desde el momento en que se crea el archivo `.service.ts`**. No se acepta construirlo primero y aplicar el patrón después.

Checklist obligatorio:

#	Requisito
1	<code>implements OnModuleInit</code> en el servicio principal
2	<code>onModuleInit()</code> ejecuta un probe ligero (ping, <code>which <cmd></code> , check de env var, query mínima)
3	Si el probe falla → <code>this.moduleHealth.registrar('<nombre>', 'degraded', '<razón>')</code> El módulo arranca igual
4	Si el probe pasa → <code>this.moduleHealth.registrar('<nombre>', 'ok')</code>
5	Los métodos que requieren el recurso externo tienen <code>assertNotDegraded()</code> o retornan <code>ModuleResult<T></code>
6	Nunca relanzar la excepción del probe fuera del <code>onModuleInit</code> — eso crashearía el backend

Módulos pendientes de construcción que DEBEN nacer degradados: IPTV/Streaming, Portal Cliente (backend de app móvil), Inventario/Almacén, pasarelas de pago adicionales, y cualquier integración futura con terceros (RENIEC, SMS, SMTP propio).

Verificación: `GET /health/modules`

D2 · El Core Indestructible: NUNCA se degrada

Enunciado: `auth`, `usuarios`, `licencia`, `clientes`, `contratos`, `planes`, `facturacion`, `pagos` (caja manual), `finanzas-opex`, `reportes`, `zonas`, `plantillas`, `config`, `schema-guard`, `auditoria`.

"Si alguno de estos falla en init → el backend **debe crashear** para proteger el servidor anterior en PM2."

Por qué: un backend que arranca a medias con el núcleo roto es peor que uno que no arranca: PM2 mantiene vivo el proceso anterior, que sí funciona.

D3 · VIO — Verified Infrastructure Operations

Checklist obligatorio para toda operación mutante nueva o modificada sobre hardware externo:

#	Requisito
1	Tras el comando de escritura, ejecutar un comando de lectura independiente que confirme el efecto esperado (el estado real del recurso, no el eco del comando)

#	Requisito
2	Si la verificación falla, el método NO reporta éxito silencioso : distingue explícitamente "aceptado, sin confirmar" de "aplicado y confirmado"
3	La verificación no bloquea indefinidamente : reintentos acotados (3–4 con backoff corto). Si el recurso puede tardar legítimamente (ej. DHCP), no fallar duro, pero dejar constancia de que no se confirmó
4	Reutilizar las funciones de verificación existentes como referencia de patrón: <code>_undo_service_port_verificado</code> , <code>check_ont_wan_pppoe</code> , <code>check_ont_mgmt_ip</code> , y los loops de <code>rollback_gpon</code> / <code>suspend_onu</code> / <code>rehabilitate_onu</code>

Alcance: código **nuevo** o que se **modifique por otra razón**. No es mandato de refactor retroactivo masivo — se corrige incrementalmente, la próxima vez que se toque cada función.

D4 · VIO hacia adentro

#	Regla
1	Todo comentario que garantice conurrencia, atomicidad o exclusión mutua ("dos instancias nunca...", "esto no puede ocurrir", "es idempotente") lleva un test que lo ejercite, o se borra . Borrarlo es una opción legítima: <i>"una garantía que nadie sostiene es peor que ninguna, porque el siguiente lector construye encima"</i>
2	Un log describe lo que ocurrió , nunca lo que el código pretendía hacer. Si el mensaje puede quedar desactualizado por un cambio en otro archivo, ya está mal escrito
3	Los tests de estas garantías nombran el incidente real que las motivó. Un test llamado "no debería fallar" se borra en la primera limpieza; uno que dice <i>"409 de lock es reintentable, no un veredicto (incidente 28/07)"</i> sobrevive

Origen del punto 2: `contratos.service.ts` logueaba *"requiere confirmación manual"* cuando el outbox ya tenía el trabajo encolado. El log describía la intención del autor al escribirlo, no el estado del sistema.

D5 · Máquina de estados declarativa

Enunciado: todo recurso con ciclo de vida contra hardware declara sus transiciones en **un solo lugar** (`domain/*-maquina-estados.ts`), no en condicionales dispersos.

#	Regla
1	La declaración indica, por transición: estados de origen legales, estado destino y qué significa en términos de negocio
2	La idempotencia se DERIVA del estado destino , no se implementa a mano en cada método: si el recurso ya está en el destino, la operación es <code>ya_en_destino</code> (ÉXITO). Un método nuevo no puede olvidarse de ser idempotente si no es él quien lo implementa
3	Los guards consultan la máquina (<code>evaluarTransicion</code>), nunca escriben su propio array
4	Un criterio disperso no es auditable; uno declarativo se revisa de un vistazo en un PR. Cualquier cambio a la lista de orígenes debe justificar por qué un estado deja de poder hacer esa transición

Origen: los estados legales de cada operación FTTH vivían en arrays y condicionales sueltos en **13 sitios**. Nadie podía leer la máquina completa y **por eso faltaba un estado de origen sin que nadie pudiera notarlo**: `desaprovisionar` no aceptaba `suspendido` , que es el caso más frecuente del negocio (un moroso suspendido al que se da de baja). Resultado: ONU huérfana.

D6 · Vocabulario de dominio (ResultadoOperacion)

Enunciado: todo método invocable por un orquestador automático devuelve `ResultadoOperacion` (`common/domain/resultado-operacion.ts`), **no excepciones HTTP**. El transporte traduce en el borde (`traducirAHttp` en el controller), nunca al revés.

Clase	Significado
<code>aplicado</code>	Se ejecutó y se verificó
<code>ya_en_destino</code>	Ya estaba así. Es ÉXITO
<code>no_aplica</code>	La operación no corresponde a este recurso
<code>rechazado_definitivo</code>	Nunca va a funcionar. No reintentar
<code>reintentable</code>	Vuelve luego
<code>indeterminado</code>	No se sabe si se aplicó

Las cuatro reglas del clasificador:

1. `indeterminado` **es obligatorio ante un timeout contra hardware**. Un timeout **NO** significa "no pasó nada": la operación pudo aplicarse y solo tardar más que el límite del cliente. No se reintenta a ciegas ni se reporta como fallo al operador — se reporta como "aceptado, sin confirmar" y se audita.
2. **La lista de rechazos definitivos es explícita y corta: solo 400 y 404**. Un criterio amplio tipo `status < 500` es **incorrecto** — 409/408/429 significan "vuelve luego".
3. **Ante la duda: reintentable**, porque reintentar es recuperable y descartar no.
4. **Nunca inferir reintentabilidad desde un código de estado HTTP**.

Origen: el outbox terminó haciendo arqueología sobre códigos HTTP y se equivocó dos veces: un no-op idempotente leído como fallo (**1.788 reintentos contra el MA5800 en 4 días**) y un 409 de lock leído como veredicto definitivo (**trabajo bueno descartado**).

D7 · Wizards y modales: un procedimiento no terminado se anula por completo

Enunciado: ningún wizard o modal que se cierre —por el motivo que sea: botón X, Cancelar, ESC, click fuera, navegación, recarga, cierre de pestaña, **crash del navegador, pérdida de sesión**— puede dejar procesos pendientes. Todo lo que se ejecutó dentro debe anularse.

Origen: incidente 2026-07-21 (CNT-2026-000004). Un wizard de provisión FTTH cerrado a medias dejó la ONU registrada en la OLT **sin `ftth_onu_registro`** , y una tarea async del carril TR-069 siguió corriendo contra un contrato que ya no tenía registro. Resultado: ONU huérfana — discordancia entre el plano físico y el lógico.

La frontera de confirmación

La frontera es el ESTADO TERMINAL VERIFICADO, no el clic del operador.

Un procedimiento está confirmado cuando su recurso alcanzó el estado terminal de su máquina de estados con verificación VIO (en FTTH: `estado = activo`). Todo lo anterior (`pendiente` , `gpon_registrado` , `wan_inyectado` , `fallido_*`) es **trabajo en vuelo y se anula al cerrar**.

Lo confirmado JAMÁS se anula por un cierre. Para deshacerlo existe la desaprovisión formal, que pide confirmación y queda auditada.

El clic NO puede ser la frontera transaccional, por dos razones:

- 1. **Es inalcanzable justo en los peores casos** —crash del navegador, caída de sesión, corte de luz— que son precisamente los que motivan esta regla. *Una frontera que no existe en el caso que la justifica no es una frontera.*
- 2. **Convertiría la regla en fábrica de cortes de servicio:** provisión correcta → ONU activa → cliente navegando → crash → el ERP desaprovisiona a un cliente en producción. La regla nació para evitar la discordancia físico↔lógico; así la crearía.

Nunca se interrumpe una operación de hardware a mitad

"Anular no es abortar: si hay una operación en vuelo contra la OLT/MikroTik, se **ESPERA** a que termine de forma atómica y recién entonces se revierte por completo. Cortar a mitad de un comando es justamente lo que deja el plano físico sucio."

Checklist obligatorio

#	Requisito
1	Ruta de anulación completa invocada en TODOS los caminos de cierre — no solo en "Cancelar"
2	El fire-and-forget debe ser cancelable. Una tarea en vuelo no puede sobrevivir a la muerte del wizard
3	Red de seguridad del lado servidor, porque el cierre puede ser un crash: marca de "wizard en curso" con dueño y heartbeat/TTL, más un barrido
4	Anular = revertir el hardware Y liberar los recursos reservados, respetando la atomicidad: nunca borrar el registro con la OLT sucia
5	Prohibir operaciones concurrentes sobre el mismo contrato/ONU. El lock cubre UNA operación; NO se toma por toda la sesión del wizard (bloquearía el contrato a los watchers)
6	La compensación se registra ANTES de ejecutar el paso (write-ahead). Orden obligatorio: escribir paso <code>en_vuelo</code> → ejecutar → marcar <code>aplicado</code> . Un paso que queda <code>en_vuelo</code> tras el TTL es SOSPECHOSO de haberse ejecutado : se verifica contra el hardware antes de decidir
7	Cada paso guarda DOS cosas: cómo deshacerlo y cómo verificar si llegó a aplicarse. Sin la sonda, un paso <code>en_vuelo</code> no es resoluble
8	Las compensaciones son idempotentes. "Does not exist" al deshacer = hecho , no error
9	VIO también al deshacer. Una compensación no confirmada NO se reporta como hecha: pasa a <code>anulacion_fallida</code> y lo hereda el watcher
10	El heartbeat SUPRIME el barrido, nunca autoriza nada, y tiene TECHO ABSOLUTO : pasado un máximo duro, el barrido procede aunque el heartbeat siga latiendo (si no, una pestaña olvidada bloquea el recurso para siempre). El servidor es la autoridad: <code>beforeunload</code> no puede ejecutar trabajo asíncrono fiable
11	Anular es asíncrono. Si el operador cierra mientras corre un paso, la anulación no es la respuesta a ese request

D8 · Los cuatro invariantes del compensador ⚠ SOLO EN CÓDIGO

Declarados en `services/compensador-wizard.service.ts` , no en `CLAUDE.md` :

#	Invariante	Justificación textual
1	Orden LIFO	"No es estética: el paso de hardware (<code>olt_gpon</code>) se registra DESPUÉS del registro y los pools, así que al invertir se limpia la OLT ANTES de soltar el registro y los IDs. Es el invariante de atomicidad expresado como orden."
2	Se detiene al primer fallo	"Continuar sería borrar el registro y liberar los pools con la OLT todavía sucia — exactamente la receta del ONT huérfano."
3	Idempotencia	""Ya no existe' al deshacer cuenta como ÉXITO, no como error."
4	VIO al deshacer	" <code>rollback_gpon</code> verifica con <code>display ont info</code> que el ONT ya no está — por eso sirve además como sonda para los pasos <code>en_vuelo</code> : ejecutarlo es a la vez comprobar y deshacer."

D9 · Reglas obligatorias de todo adaptador de protocolo ⚠ SOLO EN CÓDIGO

Declaradas en `olt-nativo/interfaces/olt-provider.interface.ts` :

" `IoltProvider` es el contrato que `NativoSshProvider` , `SmartoltProvider` y `AdminOltProvider` deben cumplir sin excepción. "

#	Regla
1	Nunca propagar una excepción al llamador. Todo error se captura internamente y se retorna como <code>OltOperacionResult { exitoso: false, mensaje, latenciaMs }</code>
2	Medir latencia con <code>Date.now()</code> al inicio y al final, incluyendo el tiempo de conexión SSH/HTTP
3	No modificar el estado de la BD desde dentro de un proveedor. Los proveedores son adaptadores puros de protocolo ; el estado lo actualiza el Router y el CircuitBreaker

Complemento: las credenciales descifradas "*nunca se persisten; viven solo en memoria durante la operación*" y **nunca se loguean**.

D10 · Aprovisionamiento multicanal ⚠ SOLO EN CÓDIGO

Declarada en `capability/cpe-provisioning-catalog.ts` :

"Un canal representa una **ESTRATEGIA** de bootstrap, no una tecnología concreta. Así, el día que Huawei use DHCP Option 43, ZTE Option 125 y VSOL otro Vendor-Specific, todos siguen siendo la MISMA estrategia (`dhcp_bootstrap`) — no hay que renombrar nada."

Regla	Enunciado
Prioridad de canal	OMCI (1) → TR-069 (2) → Option 43 (auxiliar) → HTTP (excepcional). NUNCA depender de Option 43 como diseño
Verificación	"El resolver NUNCA confía en el <code>success</code> del canal: verifica convergencia real contra GenieACS (VIO: <code>accepted ≠ materialized</code>)"
Modelo desconocido	"Un modelo no catalogado ⇒ <code>CPE_MODEL_NOT_SUPPORTED</code> , jamás un intento a ciegas "
Certificación	La decisión de canal es POR MODELO , nunca una dependencia global de un mecanismo

Lección metodológica registrada (2026-07-29):

"Una prueba de bootstrap sobre un equipo que YA tiene la configuración que se quiere ver aparecer **no prueba nada**, y su falso positivo es más caro que un falso negativo."

D11 · Portabilidad multi-VPS

Enunciado: ningún archivo del repositorio puede contener IPs, dominios o URLs de servidor hardcodeadas.

#	Regla
1	Variables de entorno, nunca literales. Todo lo que cambie entre instalaciones va en <code>.env.production</code> de cada VPS — nunca en código ni en <code>ecosystem.config.js</code>
2	Lazy getters para constantes de módulo. Las constantes top-level en servicios NestJS se evalúan antes de que <code>ConfigModule</code> lea el <code>.env</code> . Si el valor viene de <code>process.env</code> , conviértelo en función
3	<code>ecosystem.config.js</code> sin IPs. Solo lo que no cambia entre servidores (<code>NODE_ENV</code> , <code>PORT</code> , <code>RUN_CRONS</code> , límites de memoria)
4	Scripts generados dinámicamente. Los scripts MikroTik, comandos CLI y endpoints enviados a hardware se construyen llamando a los getters en tiempo de ejecución
5	<code>.env.example</code> como contrato. Toda variable nueva se documenta ahí con un comentario que explique qué valor poner. Es la guía de instalación de un servidor nuevo

Checklist antes de commit con cualquier URL o IP:

- ☐ ¿El valor viene de `process.env`?
- ☐ Si es constante de módulo, ¿es un lazy getter?
- ☐ ¿`ecosystem.config.js` sigue sin IPs ni dominios?
- ☐ ¿`.env.example` documenta la variable?

D12 · Causa raíz: checklist antes de dar por corregido un fallo

#	Requisito
1	Reproducirlo y observarlo , no deducirlo. Si el diagnóstico no se apoya en una medición —una consulta, un header, un valor leído en ejecución— es una hipótesis , y debe decirse así
2	Explicar por qué el sistema llegó a ese estado. Si la explicación es "no sé por qué pasaba, pero con esto ya no pasa", no está corregido: está oculto
3	Preguntar dónde más ocurre lo mismo. Un defecto de criterio suele estar repetido
4	Corregir en el punto común , no en cada sitio donde se manifiesta
5	Dejar constancia de la causa , no del arreglo. El commit y el comentario explican qué estaba mal y por qué no se veía

D13 · Reutilizar: checklist antes de escribir una consulta

#	Requisito
1	Buscar primero. Un <code>grep</code> por el concepto de negocio cuesta un minuto
2	Si existe y sirve, usarlo. Aunque devuelva de más: filtrar sobra es barato, mantener dos fuentes no

#	Requisito
3	Si existe pero no encaja, extenderlo , no clonarlo
4	Si de verdad hace falta uno nuevo, dejar UNA definición reutilizable (constante, CTE, servicio)
5	Justificar la duplicación cuando sea inevitable , por escrito en el código: qué se duplicó, por qué, y qué hay que cambiar en los dos sitios si cambia la regla

PARTE III — POLÍTICAS DE DOMINIO

P1 · Ciclo de vida de las IPs VPN

Política	Enunciado
Permanencia	Las IPs VPN son permanentes. Una vez que un cert conecta y recibe su IP, esa IP queda bloqueada en el CCD del servidor (<code>ifconfig-push</code>) para ese cert. OpenVPN nunca debe reasignar esa IP a otro equipo
Liberación	Solo en dos casos: (1) el router se elimina → <code>removeRouter</code> revoca los certs → se elimina el CCD → se mata el túnel; (2) el wizard se cierra sin completar el paso 3 → <code>fireRevoke</code>
Túnel	Al eliminar un router o cancelar el wizard, el túnel debe eliminarse (<code>revocar</code> → <code>killClienteVpnManagement</code> + borrar CCD). No dejarlo activo
Reutilización	Nunca reutilizar un cert que ya tenga <code>vpnIp</code> asignada sin verificar primero que esa IP no esté en uso por otro router activo en BD
Red de seguridad	Cron <code>limpiarWizardsAbandonados</code> (cada 30 min, corte a 2 h)

P2 · El `iroute` declara PROPIEDAD, no alcanzabilidad

Nunca ampliar el CCD de un router para "poder llegar" a la red de otro.

El CCD sale de `subnets_locales` y es lo que **atribuye clientes**. A nivel OSPF todo se alcanza y está bien; a nivel ERP el modelo es de **propiedad**.

"Dos routers reclamando la misma red **no falla ruidosamente**: da la respuesta equivocada con naturalidad."

P3 · Un script VPN por wizard, nunca regenerable

La edición solo permite **visualizar** el script original. No se regenera.

P4 · Migraciones de ONUs — advertencia obligatoria

Antes de diseñar, programar o ejecutar cualquier migración que incorpore ONUs existentes al ERP (SmartOLT, MikroWISP, adopción masiva de huérfanas), leer esto y actuar en consecuencia. No es opcional.

El mecanismo: una ONU con `contrato_onu_config.provisioning_enabled = true` y `last_applied_revision` en NULL queda marcada como **drift**, y el reconcile de las 03:30 le **reescribe el SSID, la clave del WiFi y las credenciales de acceso web** con el preset de la OLT.

"En una migración eso no afecta a una ONU: afecta a **todas a la vez, de madrugada, sin que nadie lo pida**. Son clientes reales en producción que llevan años con su configuración —muchos con su propia clave de WiFi—. A la mañana siguiente, ninguno tiene internet en sus dispositivos y nadie sabe por qué."

Estado actual: el ERP está a salvo **por construcción, no por precaución**. Hay 205 ONUs en la OLT y solo las que el ERP aprovisionó tienen `contrato_onu_config` . `adoptarOnusHuerfanas` inserta solo en `ftth_onu_registro` y NO crea config. **La migración es exactamente el proceso que rompería esa garantía.**

Reglas para cualquier migración de ONUs:

#	Regla
1	Una ONU migrada/adoptada nace con <code>provisioning_enabled = false</code> , o con <code>last_applied_revision = revision</code> para que nunca figure como drift
2	El auto-config solo se aplica a aprovisionamientos nuevos hechos desde el ERP. Una ONU que ya funcionaba se ADOPTA (se observa y se respeta), nunca se reconfigura
3	Antes de activar el reconcile sobre el parque migrado, verificar el conteo de drift. Si devuelve cientos, la madrugada siguiente es un incidente

```
SELECT COUNT(*) FROM contrato_onu_config
WHERE provisioning_enabled
  AND (last_applied_revision IS NULL OR last_applied_revision < revision);
```

P5 · La frontera del dinero

Declarada en `pagos/adaptadores/README.md` .

Por qué el contrato existe pero los adaptadores no

"El contrato se fijó en la Etapa I **a propósito**. Si se hubiera dejado para la Etapa II, la primera integración lo habría definido **de facto** y las demás se habrían acomodado a las peculiaridades de ese proveedor."

La puerta de estabilidad

Los adaptadores no se construyeron porque la Etapa II tiene una puerta: **30 días de invariante de contabilidad limpio en producción, un extorno real revisado a mano y un cierre de caja mensual cuadrado**. Dos de esos tres criterios **no dependen de escribir código**.

"No es burocracia. Cada integración que se apile sobre una frontera no demostrada multiplica el coste de descubrir que la base estaba mal, y ese descubrimiento llega con **dinero de clientes en juego**."

Orden obligatorio antes del primer adaptador

1. **Comprobar la puerta** (los cinco criterios del plan).
2. **Construir el motor primero** (`cobro_intento` + conciliador). "Un adaptador sin la máquina de estados del cobro en vuelo no tiene dónde reportar un `indeterminado` , y entonces alguien lo va a reportar como fallo — que es la decisión incorrecta."
3. **Migrar Mercado Pago al contrato antes que ningún proveedor nuevo**. "Es el único que ya cobra dinero real: si la abstracción no lo absorbe, la abstracción está mal y **se corrige con un proveedor, no con tres**."

Las tres cosas que ya salieron mal, y qué las bloquea

Error	Qué pasó	Qué lo bloquea
Un segundo servicio que registra pagos	El registro ya existe (<code>PagosService.registrar</code>). Un paralelo nace sin el reconciliador ni los guards	<code>frontera-dinero.spec.ts</code>
Aplicar dinero fuera del aplicador	Había 4 copias del mismo <code>UPDATE</code> ; la de <code>adelantos</code> había perdido el guard de estado y aplicaba saldo a favor contra facturas ANULADAS	<code>frontera-dinero.spec.ts</code>
Inferir reintentabilidad de un código HTTP	Un 409 de lock se leyó como veredicto definitivo y se descartó trabajo bueno; un no-op idempotente se leyó como fallo → 1.788 reintentos contra el MA5800 en 4 días	<code>contrato-adaptador.spec.ts</code> , <code>resultado-operacion.spec.ts</code>

La regla que se olvida siempre

"**Un timeout cobrando NO significa 'no pasó nada'**. Al cliente pudo cobrársele y la respuesta perderse. Reintentar a ciegas le cobra dos veces; reportar fallo deja dinero existiendo sin registro. **Las dos opciones que parecen simples son las dos incorrectas**: se reporta `indeterminado` y lo resuelve el conciliador consultando al proveedor."

P6 · Ciclo de cobro: una sola fórmula

Una sola fórmula, en `politica-facturacion.service.ts` . La **gracia es la distancia vencimiento→corte**; **NO se suma al vencimiento**.

Origen: antes eran tres fórmulas y el corte llegaba a caer **antes** del vencimiento (incidente 05/08, cliente James Pena).

P7 · Usar el flujo de negocio, nunca SQL directo

Enunciado: un `UPDATE` directo **se salta las cascadas** (revocar certs VPN, quitar rutas, invalidar pool).

Origen: 28/07 — pasó, y dejó un cert huérfano reservando una IP.

Corolario: "el invariante que solo vive en la doc no es invariante" — por eso se codificó en `verificarInvariantes()` .

P8 · Políticas de pools de recursos **△ SOLO EN CÓDIGO**

`services/olt-mgmt-ip-pool.service.ts` :

"**NUNCA retira una IP OCUPADA**: esa IP está escrita en el IP-host de una ONU viva. Sacarla del pool dejaría al ERP sin saber que le pertenece, y el tramo podría reasignarse a otra OLT: **dos ONUs con la misma IP en el mismo L2**. Para retirar una IP ocupada primero hay que desaproveionar o desactivar el carril de esa ONU, que es lo que la libera de verdad."

Y sobre por qué existe el retiro: "Sin esto solo se podían **AÑADIR** IPs, y la única salida era un `UPDATE` a mano contra la tabla — justo lo que el ERP no debe requerir."

P9 · Baseline versionado inmutable ⚠ SOLO EN CÓDIGO

`services/olt-baseline.service.ts` :

"Versionado inmutable: crear con un nombre existente genera `version = max+1` . **Nunca se edita una versión publicada** — el historial de qué se exigió en cada momento queda auditable."

Y en `capability/olt-baseline-standard.ts` , una regla de diseño de VLANs:

"VLAN 220 ERP-IPTV — **RESERVADA**: se agrega al spec cuando exista el módulo IPTV (**no crear VLANs sin consumidor**)."

P10 · Una OLT admite UN SOLO proveedor ⚠ SOLO EN CÓDIGO

`olt-nativo.service.ts` y migración `OltUnProveedorPorOlt` :

"Regla de negocio: una OLT admite UN SOLO proveedor (el que se fija al registrarla)."

P11 · Regla crítica de caída de VPN ⚠ SOLO EN CÓDIGO

`olt-monitoreo.service.ts` :

"Si Python no responde → marcar OLT como OFFLINE, **NO tocar ONUs**. Se emite UNA alerta crítica global. Los estados se **congelan**."

Y la regla hermana: "Regla estricta: **NO modificar estados de ONUs**" desde el monitoreo.

Por qué importa: perder el canal de observación no es lo mismo que observar que algo cayó. Confundirlos convertiría un corte de VPN en un apagón masivo aparente de todo el parque.

P12 · Invariante físico de la planta externa ⚠ SOLO EN CÓDIGO

Migración `CreatePlantaExternaOptica` :

"INVARIANTE FÍSICO: un hilo se fusiona **UNA sola vez**, por cada extremo."

P13 · Correlativos sin `MAX()+1` ⚠ SOLO EN CÓDIGO

`entities/comprobante-config.entity.ts` :

"Nunca usar `MAX()+1` para evitar race conditions."

P14 · Invariante de atomicidad hardware ↔ ERP

Nunca un `ont` en la OLT sin `ftth_onu_registro` , ni al revés.

Sostenido por dos watchers:

Watcher	Dirección
<code>reintentarRollbacksFallidos</code> (estado <code>fallido_rollback</code>)	DELETE — nunca borrar el registro con la OLT sucia
<code>adoptarOnusHuerfanas</code>	CREATE — reconstruye desde inventario + pool + lectura viva de la OLT (VIO)

La descripción del `ont` en la OLT lleva `DATAFAST_CNT-xxxx` para poder atribuirlo.

PARTE IV — POLÍTICAS DE OPERACIÓN E INFRAESTRUCTURA

Declaradas en `ecosystem.config.js`, que se autodefine como **fuentes de verdad única**.

01 · El arranque se declara, no se improvisa

"Antes estaban repartidos entre tres archivos —uno de ellos SIN VERSIONAR— y lo que realmente corría **no coincidía con lo declarado en el repo**: una instalación nueva no era reproducible."

#	Regla
1	Cualquier cambio de arranque se hace AQUÍ y se despliega; nunca con <code>pm2 start</code> manual
2	Prohibido poner IPs, dominios o secretos: van en los <code>.env</code> de cada VPS
3	Las apps de backend NO reciben credenciales por <code>env</code> : las lee la aplicación desde <code>.env.production</code> . PM2 solo declara lo que distingue a cada proceso (rol, puerto, límites)

02 · Un solo proceso migra

"`api-core` y `worker` arrancan a la vez y competían por las migraciones (2026-07-21, `duplicate key ... pg_type_typname_nsp_index`). Fue inofensivo por ser idempotente, pero **una migración menos defensiva puede dejar el esquema a medias**."

El default es `true` **a propósito** (retrocompatible): un VPS que aún no declare la variable se comporta como hasta ahora.

03 · Aislamiento de procesos por causa real

Proceso	Política	Origen
<code>datafast-whatsapp</code>	Único proceso que aloja Chromium. "Si Chromium se descontrola muere solo, sin arrastrar ni la API ni el outbox de red"	El worker llegó a dejar el VPS con 87 MB libres
<code>datafast-frontend</code>	Entorno MÍNIMO a propósito	Hasta 2026-07-22 arrastraba todos los secretos (<code>DB_PASSWORD</code> , <code>ENCRYPTION_KEY</code> , <code>JWT_SECRET</code> , <code>REDIS_PASSWORD</code>) por haberse lanzado desde una shell con el <code>.env</code> del backend. "El frontend es el proceso expuesto y no necesita ninguno"
<code>olt-automation-service</code>	1 worker, NUNCA <code>--reload</code>	Ver O4

04 · Nunca `--reload` en producción

"WatchFiles reinicia uvicorn al tocar cualquier archivo y un `git reset --hard` de deploy lo dispara **en medio de una operación contra la OLT**. Causó el timeout que abortó una Fase 2 WAN y dejó un ONT huérfano (2026-07-21)."

"**1 worker a propósito:** cada worker abre sus propias sesiones SSH y el MA5800 tiene un límite bajo de VTY concurrentes."

05 · Timeouts realistas contra hardware

Operación	Antes	Ahora
rollback-gpon	30 s	150 s
inject-wan-pppoe	30 s	90 s

Origen: el de 30 s causó el ONT huérfano del 21/07 a las 16:55.

Regla derivada: "un timeout puede significar que la operación Sí se aplicó."

06 · Ningún dominio es obligatorio

Declarado en `docker-compose.yml` :

"Tres roles, tres hosts, **ninguno obligatorio:** el ERP no está atado a ningún dominio ni IP. Una instalación puede servirse por IP a secas, en una LAN, o con los tres nombres — y ninguna de las tres es un caso especial en el código."

Regla	Enunciado
Retrocompatibilidad al renombrar	" <code>ERP_DOMAIN</code> cae en <code>APP_DOMAIN</code> si no está definido: renombrar una variable sin periodo de gracia rompe toda instalación existente en su próxima actualización"
Ausencia elegante	"Sin <code>WEB_DOMAIN</code> el vhost queda con un <code>server_name</code> inalcanzable en vez de desaparecer: nginx no admite plantillas condicionales, y un vhost que nadie resuelve es inofensivo "

07 · Nunca `synchronize` en producción ⚠ SOLO EN CÓDIGO

`config/database.config.ts` : "NUNCA usar `synchronize` en producción — usar migraciones."

Y en `datasource.install.ts` : "Regla: instalación nueva → `migration:run:all` . Deploy incremental → como está."

08 · Los crons nunca relanzan ⚠ SOLO EN CÓDIGO

`ztp-reconcile.cron.ts` : "Nunca relanzar desde un cron: **tumbaría el proceso PM2.**"

09 · Registro de deuda técnica

`PENDIENTES.md` se autodefine así:

*"Registro vivo de lo que queda por hacer. **Cualquier sesión debe leer este archivo cuando se pregunte '¿qué pendientes hay?'**, y añadir aquí lo nuevo que quede abierto **en vez de dejarlo solo en el mensaje de una conversación que nadie volverá a leer.**"*

Formato obligatorio de cada entrada: qué falta, **por qué importa** (la consecuencia real, no la tarea) y cómo se comprueba.

"Una entrada sin consecuencia acaba siendo ignorada."

O10 · Credenciales

Ubicación	Política
ACCESOS.local.md	Todas las credenciales del entorno. Solo local — nunca a GitHub ni al VPS
.env.production	Secretos de infraestructura, en el filesystem del VPS
Base de datos	Credenciales de equipos y terceros, cifradas (encryption.util)
Frontend	Ningún secreto

O11 · Workspace de scripts de red

Los scripts MikroTik van en Proyecto_CRM_ISP/mikrotik-network/ , nunca dentro de erpdatafast-isp/ .

O12 · Nunca tocar el balanceo de OASIS sin leer antes

Regla crítica: nunca modificar el balanceo de OASIS sin leer mangle + rutas + address-lists. El sistema usa Linea1-8 por src-address-list , no PCC.

PARTE V — DIRECTRICES DE TRABAJO DEL EQUIPO

Del `CLAUDE.md` global. Definen el rol y el método, no el código.

Rol permanente

Arquitecto de Software Principal e Ingeniero Jefe de Redes, especializado en Telecomunicaciones, automatización ISP y sistemas de alta disponibilidad.

Perfiles por área

Al trabajar en...	Rol que se asume
<code>backend/src</code> /	Ingeniero Senior en NestJS y microservicios — DI estricta, tipado fuerte, optimización TypeORM, excepciones robustas
Scripts de red	Ingeniero de Redes (MikroTik MTCNA/MTCRE, WISP/FTTH) — estabilidad de sockets, control de concurrencia en RouterOS API, logs preventivos
<code>frontend/src</code> /	Arquitecto Frontend en Next.js 14 App Router — SSR y estados globales limpios con Zustand

Evaluación pesimista obligatoria

Antes de cualquier solución, evaluar internamente el **peor escenario posible**:

#	Pregunta
1	Latencia/Timeouts: ¿qué pasa si un OLT o MikroTik cae a mitad del provisioning? → rollbacks, reintentos, estados transaccionales
2	Race conditions: ¿qué pasa si dos usuarios provisionan el mismo ONU o asignan el mismo perfil PPPoE simultáneamente?
3	Desfase de estado: ¿qué pasa si la BD dice "activo" pero el OLT reporta "rogue" o "desconectado"?
4	Input: sanitizar todas las variables enviadas a CLIs de hardware o APIs de RouterOS

Estándares de código

Estándar	Enunciado
Clean Code	SOLID, DRY, arquitectura limpia
Diseño cero-error	Manejo robusto de errores, excepciones tipadas, degradación elegante, logging comprensivo. Sin placeholders, sin TODOs, sin <code>catch</code> genéricos sin lógica de recuperación
Listo para producción	Código completo, optimizado, seguro contra OWASP Top 10
Idioma	Responder y documentar siempre en español
Tipado	Estricto en TypeScript — evitar <code>any</code>

Método de trabajo

#	Directriz
1	No programar sin contexto. Leer los archivos relevantes y el <code>git log</code> antes de escribir. Si no hay contexto suficiente, preguntar; no asumir
2	Inspección integral obligatoria antes de cualquier solución: código local → VPS → ERP → routers/antenas si aplica
3	Validar antes de declarar hecho. Compilar, correr tests o verificar. Nunca decir "listo" sin evidencia de que funciona
4	Soluciones simples. Lo mínimo que resuelve el problema. Sin abstracciones, helpers, tipos ni validaciones que no se pidieron. 3 líneas repetidas > 1 abstracción prematura
5	No reescribir archivos completos. Cambiar solo lo necesario; no "limpiar" el código de alrededor
6	No pelear con el usuario. Si discrepas, menciona el concern en una oración y procede — salvo riesgo real de seguridad o pérdida de datos
7	Corrección definitiva, no parches: contexto completo, peor escenario, causa raíz, verificación con evidencia real
8	Commit y deploy tras cada cambio sin errores

PARTE VI — INVARIANTES DEL SISTEMA

La lista canónica de lo que **nunca puede ser falso**, con su mecanismo de garantía. La última columna es la que importa: distingue lo garantizado de lo confiado.

#	Invariante	Garantizado por	Verificado
1	Nunca un <code>ont</code> en la OLT sin <code>ftth_onu_registro</code> , ni al revés	2 watchers (<code>reintentarRollbacksFallidos</code> , <code>adoptarOnusHuerfanas</code>)	Producción
2	Un solo escritor del saldo de una factura	<code>AplicadorFacturaService</code> + trigger <code>trg_factura_saldo</code>	<code>frontera-dinero.spec.ts</code>
3	Un solo registrador de pagos	<code>PagosService.registrar</code>	<code>frontera-dinero.spec.ts</code>
4	El extorno es la única reversión legítima de un pago	<code>pago_extorno</code>	<code>extorno.spec.ts</code>
5	Una sola fórmula del ciclo de cobro	<code>PoliticaFacturacionService</code>	<code>politica-facturacion.service.spec.ts</code>
6	Dos instancias PM2 nunca ejecutan el mismo comando de red	Reclamo atómico <code>EN_PROCESO</code> + dueño + TTL	<code>outbox-red.claim.spec.ts</code>
7	Solo 400 y 404 son rechazos definitivos	Clasificador del outbox	<code>resultado-operacion.spec.ts</code> , <code>contrato-adaptador.spec.ts</code>
8	Toda transición ilegal se rechaza; toda repetida es <code>ya_en_destino</code>	<code>ftth-maquina-estados.ts</code>	<code>ftth-maquina-estados.spec.ts</code>
9	Un token de portal no accede a otro tenant	<code>PortalAuthGuard</code> + <code>PortalTenantService</code>	<code>portal-auth.aislamiento.spec.ts</code>
10	La IP VPN de un router es permanente hasta su baja	CCD con <code>ifconfig-push</code> escrito en el primer handshake	Producción
11	Un hilo de fibra se fusiona una sola vez por extremo	Restricción en BD	Migración
12	Nunca dos ONUs con la misma IP de gestión en el mismo L2	El pool no retira IPs ocupadas	⚠ Solo por código
13	Una OLT tiene un solo proveedor	Índice + guard	Migración
14	Un router activo, una IP de gestión por empresa	<code>uq_routers_empresa_ip_gestion</code> WHERE <code>activo</code>	Índice
15	Una ONU por contrato	<code>uq_contratos_empresa_onu</code>	Índice
16	Una notificación no se envía dos veces	<code>idx_notif_logs_idempotency_key</code> UNIQUE	Índice
17	Nunca se edita una versión publicada de un baseline	<code>OltBaselineService</code>	⚠ Solo por código
18	Una caída de VPN no altera el estado de ninguna ONU	<code>OltMonitoreoService</code>	⚠ Solo por código

#	Invariante	Garantizado por	Verificado
19	El aislamiento entre empresas	<code>empresa_id</code> en cada consulta	⚠ Solo por convención
20	El reconcile no toca ONUs preexistentes	<code>adoptarOnusHuerfanas</code> no crea config	⚠ Solo por efecto lateral

Los invariantes 19 y 20 son los únicos de la lista que no tienen mecanismo propio. El 19 depende de que 445 consultas se acuerden de filtrar; el 20, de un efecto lateral que nadie declaró como garantía. Son, respectivamente, el riesgo de fuga entre tenants y el riesgo de reescritura masiva de configuración de clientes.

PARTE VII — CATÁLOGO DE LECCIONES

Cada regla de este documento tiene un incidente detrás. Esta es la correspondencia.

Fecha	Incidente	Regla que nació
2026-07-12	La EG8145V5 adopta la ACS URL por DHCP; OMCI no la escribe	Bootstrap multicanal por estrategia, no por tecnología
2026-07-14	OLTs y routers duplicados	Índice UNIQUE parcial por <code>(empresa_id, ip_gestion)</code> <code>WHERE activo</code>
2026-07-15	—	Implementación desde cero: el ERP inyecta su config, nunca se adapta
2026-07-17	CNT-2026-000004: la ONU aceptó el comando OMCI y nunca lo materializó	VIO: <code>accepted ≠ materialized</code>
2026-07-18	Reboot/WiFi no funcionaban desde TR-069	Provision <code>erp-connreq-creds</code> ; credenciales deben coincidir con el <code>.env</code> de cada VPS
2026-07-19	Gestión TR-069 resuelta end-to-end	<code>ont reset</code> de OLT no gatilla boot-inform; el power-cycle físico sí
2026-07-21 16:55	Timeout de 30 s abortó una Fase 2 WAN → ONT huérfano	Timeouts realistas (90 s / 150 s); un timeout puede significar que SÍ se aplicó
2026-07-21	Wizard cerrado a medias → ONU huérfana	Wizards: lo no confirmado se anula. Saga con bitácora write-ahead
2026-07-21	Dos procesos compitiendo por las migraciones	Solo <code>api-core</code> migra
2026-07-21	<code>--reload</code> de uvicorn disparado por un <code>git reset --hard</code> de deploy	Nunca <code>--reload</code> en producción
2026-07-28	Un <code>UPDATE</code> directo dejó un cert VPN huérfano reservando una IP	Usar el flujo de negocio, nunca SQL directo. El invariante que solo vive en la doc no es invariante
2026-07-28	El comentario del outbox garantizaba una exclusión mutua que era FALSA	VIO hacia adentro: un comentario que garantiza concurrencia lleva un test, o se borra
2026-07-28	Un 409 de lock leído como veredicto; un no-op leído como fallo → 1.788 reintentos en 4 días	Solo 400 y 404 son definitivos. Nunca inferir reintentabilidad de un código HTTP
2026-07-28	Baja imposible desde <code>suspendido</code> → ONU huérfana	Máquina de estados declarativa en un solo archivo
2026-07-29	El "bucle" de la ONU eran dos latencias encadenadas (287 s → 8 s)	El segundo defecto solo se ve tras corregir el primero
2026-07-29	Se intentó promover OMCI a CERTIFIED y se revirtió	Probar bootstrap sobre un equipo que ya tiene la config buscada da falso positivo
2026-07-30	Chromium en el worker → VPS con 87 MB libres	Chromium en su propio proceso PM2
2026-07-22	El frontend arrastraba todos los secretos del backend	Entorno PM2 mínimo para el proceso expuesto
—	Factory-reset dejaba la gestión TR-069 muerta	No era IP ni Option 43: era el tag <code>AuthEnforced</code> con la ONU informando sin credenciales
—	Router zombi: la interfaz <code>vpndatafast</code> reintentando cada 15 s para siempre	La baja de un router limpia el cliente en el MikroTik

Fecha	Incidente	Regla que nació
2026-08-05	El mapa: 3 fallos con 3 parches superficiales disponibles	Causa raíz antes que parche. Corregir en el punto común (CTE PUNTOS_SERVICIO)
2026-08-05	Tres fórmulas del ciclo de cobro; el corte caía antes del vencimiento	Una sola fórmula. La gracia es la distancia vencimiento→corte
2026-08-06	El backend estuvo 11 h ejecutando código viejo; el deploy afirmaba éxito sin comprobarlo	Una verificación que solo sabe confirmar el caso bueno no es una verificación
—	4 copias del UPDATE que aplica dinero; una aplicaba a facturas anuladas	Un solo escritor del saldo, protegido por test

PARTE VIII — CARTA MAGNA

Los diez principios que resumen todo lo anterior. Si algún día hay que elegir qué conservar de este documento, es esto.

- I. El hardware es la verdad; la base de datos es una creencia que se verifica.

II. Aceptar no es aplicar. Sin lectura independiente que lo confirme, se reporta "aceptado, sin confirmar" — nunca "hecho".

III. Un timeout no significa que no pasó nada. Es `indeterminado` : ni se reintenta a ciegas ni se reporta como fallo.

IV. Reintentar es recuperable; descartar no. Ante la duda, reintentable. Solo 400 y 404 son definitivos.

V. Lo no confirmado se anula por completo; lo confirmado jamás se anula por un cierre. La frontera es el estado terminal verificado, nunca el clic.

VI. Nunca se interrumpe una operación de hardware a mitad. Anular no es abortar.

VII. El invariante que solo vive en la documentación no es un invariante. Si nadie lo sostiene con un mecanismo o un test, se codifica o se borra.

VIII. Un log describe lo que ocurrió, nunca lo que el código pretendía hacer.

IX. Se busca la causa, no el síntoma; y se corrige en el punto común, no donde se manifiesta. Si la explicación es "no sé por qué pasaba, pero ya no pasa", no está corregido: está oculto.

X. El ERP inyecta su configuración canónica en los equipos que provisiona, y respeta como intocable lo que encontró funcionando.

Anexo — Directrices que solo viven en el código

Las siguientes reglas **no están en `CLAUDE.md` ni en `docs/`** . Solo existen como comentarios dentro de un archivo, y se perderían si ese archivo se reescribiera sin leerlo. Se listan aquí para que dejen de depender de eso.

Directriz	Archivo
Los 4 invariantes del compensador (LIFO, parada al primer fallo, idempotencia, VIO al deshacer)	<code>services/compensador-wizard.service.ts</code>
Las 3 reglas obligatorias de todo adaptador de protocolo	<code>interfaces/olt-provider.interface.ts</code>
Un canal es una estrategia, no una tecnología · modelo no catalogado ⇒ error, jamás intento a ciegas	<code>capability/cpe-provisioning-catalog.ts</code>
La lección metodológica del falso positivo de bootstrap	<code>capability/cpe-provisioning-catalog.ts</code>
El baseline es la única fuente de verdad de la config canónica	<code>capability/olt-baseline-standard.ts</code>
No crear VLANs sin consumidor	<code>capability/olt-baseline-standard.ts</code>
Nunca se edita una versión publicada de un baseline	<code>services/olt-baseline.service.ts</code>
Nunca retirar del pool una IP de gestión ocupada	<code>services/olt-mgmt-ip-pool.service.ts</code>
Si Python no responde: OLT OFFLINE, no tocar ONUs, congelar estados	<code>olt-monitoreo.service.ts</code>
El monitoreo no modifica estados de ONUs	<code>olt-monitoreo.service.ts</code>

Directriz	Archivo
Una OLT admite un solo proveedor	<code>olt-nativo.service.ts</code> + migración
Un hilo se fusiona una sola vez por extremo	Migración <code>CreatePlantaExternaOptica</code>
Nunca <code>MAX()+1</code> para correlativos	<code>entities/comprobante-config.entity.ts</code>
Nunca <code>synchronize</code> en producción	<code>config/database.config.ts</code>
Instalación nueva vs deploy incremental	<code>config/datasource.install.ts</code>
Un cron nunca relanza (tumbaría PM2)	<code>cron/ztp-reconcile.cron.ts</code>
El motor de capacidades no muta la entrada; reglas puras	<code>capability/capability.engine.ts</code>
El orden de registro de rutas en NestJS/Express importa	<code>contratos.controller.ts</code>
La política completa de la frontera del dinero	<code>pagos/adaptadores/README.md</code>
Todas las políticas de arranque, aislamiento y despliegue	<code>ecosystem.config.js</code>
Ningún dominio es obligatorio; retrocompatibilidad al renombrar variables	<code>docker-compose.yml</code>